



FOR SPRING DEVELOPERS MOVING TO MICRONAUT

What Moves to Compile Time

Spring Boot to Micronaut, the flagship

Much more of Micronaut is generated at compile time. Bean creation can still be lazy. Those are different things.

One payment service written twice, in Spring Boot and in Micronaut, against one Postgres. The controllers, services and repositories read almost line for line. Four small mistakes do not behave the same way, and this sheet is those four, with the code each one depends on.

These are the DEFAULT modes: Spring's proxy based transactions on the normal JVM, without AOT, and Micronaut's default around advice and lazy singletons. Spring's AspectJ mode changes the self call result.

Every version on this sheet was current on 24 September 2026 and will not stay current. The method is the durable part: run the script, read what came back, and write down the result rather than the expectation.

In scope

- A typo in a derived finder: when each framework rejects it
- @Transactional on a private method
- A @Transactional method called from the same class
- A bean nobody provides, and what eager init does to it

Out of scope

- Which framework is faster. Nothing here measures throughput or startup
- Native images, and Spring AOT, which changes when Spring decides things
- Anything about how this channel is produced, which is not the subject

Spring Data JPA and Micronaut Data JDBC are not the same persistence stack. The finder comparison is about one thing only: when a finder that cannot work is rejected.

01 Where each mistake surfaces

Typo in a derived finder	Micronaut: compile. Spring: startup
@Transactional on a private method	Micronaut: compile. Spring: no framework error
@Transactional method, called from the same class	Micronaut: intercepted. Spring: no framework error
A bean nobody provides	Spring: startup. Micronaut: first request
The same, with eagerInitSingletons(true)	Micronaut: startup

Read it as: the column is the first moment the framework tells you. "No framework error" means the method runs and the transaction advice does not.

02 The finder typo: compile against startup

Both repositories derive the query from the method name. Add one letter too many and Spring builds without complaint, then refuses to start. Micronaut fails the build.

Micronaut Data is checking the method against the ENTITY MODEL at compile time: it cannot find a property called Currencyy on Payment, and lists the ones it can. It is not checking your database.

```
micronaut-payments/src/main/java/dev/codewithsam/payments/PaymentRepository.java
```

```
@JdbcRepository(dialect = Dialect.POSTGRES)
public interface PaymentRepository extends CrudRepository<Payment, Long> {

    List<Payment> findByCurrency(String currency);
}
```

```
The same interface with findByCurrencyy, on both frameworks
```

```
# Micronaut: ./mvnw compile
[ERROR] PaymentRepository.java:[11,19] Unable to implement Repository method:
...findByCurrencyy(String currency). Cannot query entity [Payment]
on non-existent property: Currencyy [amount, currency]

# Spring: ./mvnw package → BUILD SUCCESS, then java -jar:
No property 'currencyy' found for type 'Payment'; Did you mean 'currency'
```

Micronaut fails compilation on the typo; Spring builds and fails at startup.

Run on a real machine, 2026-09-24. Reproduce it with `scripts/verify.sh` in the course repository.

"Micronaut Data will actively check at compile time that a repository method can be implemented and fail compilation if it cannot."

micronaut-projects.github.io/micronaut-data, checked 2026-09-24

03 @Transactional on a private method

record() saves the payment, then rejects it for being over the limit. The rejection should undo the save, and only a transaction can do that.

Spring's default transaction mode applies the advice through a proxy. A class based proxy cannot override a private method, so the advice is never applied and nothing warns you: post an over the limit payment, get a 500, and count one row in the table.

Micronaut generates its proxy at compile time as a real subclass of your service. A subclass cannot override a private method either, so the build stops and says so.

mistakes/private-transactional: PaymentService.java (both frameworks)

```
public Long record(long amount, String currency) {
    return write(amount, currency);
}

@Transactional
private Long write(long amount, String currency) {
    Payment payment = repository.save(new Payment(amount, currency));
    if (amount > LIMIT) {
        throw new PaymentRejected(amount);
    }
    return payment.getId();
}
```

Micronaut: "Method annotated as executable but is declared private. Change the method to be non-private in order for AOP advice to be applied."

Spring: the build passes, the request returns 500, and the rejected payment is in the table.

Run on a real machine, 2026-09-24. Reproduce it with scripts/verify.sh in the course repository.

Micronaut: the build fails with the message above.

Run on a real machine, 2026-09-24. Reproduce it with scripts/verify.sh in the course repository.

04 The self call: one rollback, one written row

recordAll() calls record() inside the same class. In Spring's default mode a self call does not go back through the transaction proxy, so the advice is bypassed and the rejected payment is still written.

In Micronaut the bean IS the generated subclass, so this.record() lands on the override and the transaction applies. Rolled back.

```
javap -p on what Micronaut generated for PaymentService
```

```
class dev.codewithsam.payments.$PaymentService$Definition$Intercepted
    extends dev.codewithsam.payments.PaymentService
    implements io.micronaut.aop.Intercepted
    public java.lang.Long record(long, java.lang.String);
```

It overrides record, the method it can intercept, and nothing else.

POST /payments/batch with one over the limit payment: Spring leaves 1 row, Micronaut leaves 0.

Run on a real machine, 2026-09-24. Reproduce it with scripts/verify.sh in the course repository.

"self-invocation ... does not lead to an actual transaction at runtime even if the invoked method is marked with @Transactional." AspectJ mode changes this.

Spring Framework reference, transaction management, checked 2026-09-24

05 It started. Then the first request failed.

Checkout needs a Gateway and nothing provides one. Spring refuses to start. Micronaut starts in well under a second, and the first request to /checkout fails with a 500: no bean of type Gateway exists.

This is by design. Micronaut creates singleton beans lazily, on demand, to optimise startup. The bean definitions were generated at compile time; the beans themselves are created at run time, when first needed.

`eagerInitSingletons(true)` makes the server refuse to start instead. That does not fix anything, it moves when you find out, and it is a trade off rather than a setting every application needs. Providing a Gateway is the fix.

```
mistakes/eager-init: Application.java
```

```
public static void main(String[] args) {  
    Micronaut.build(args)  
        .eagerInitSingletons(true)  
        .mainClass(Application.class)  
        .start();  
}
```

Micronaut logged "Server Running", then POST /checkout returned 500 with "No bean of type [Gateway] exists".

Run on a real machine, 2026-09-24. Reproduce it with `scripts/verify.sh` in the course repository.

"by default @Singleton-scoped beans are created lazily and on-demand. This is by design to optimize startup time."

`docs.micronaut.io`, Micronaut 5 guide, checked 2026-09-24

Before you trust a Micronaut service in production

1. No AOP annotation on a private method; the build will tell you, so read the error rather than working round it
2. You know which calls are self calls, and that Micronaut intercepts them where Spring's default mode does not
3. A smoke test hits every endpoint after startup, because a missing bean can survive startup
4. You chose eager or lazy singletons on purpose, knowing what each costs
5. `scripts/verify.sh` passes on a fresh clone



Everything else

- | | |
|--|---|
|  Site
https://code-with-sam-dev.github.io |  GitHub
https://github.com/code-with-sam-dev |
|  YouTube
https://www.youtube.com/@CodewithSam-Dev |  Buy me a coffee
https://buymeacoffee.com/codewithsam |
|  LinkedIn
https://www.linkedin.com/company/code-with-sam-dev |  Buy me sadza
https://buymesadza.com/codewithsam |
|  X
https://x.com/CodeWithSamDev | The course repository
https://github.com/code-with-sam-dev/spring-to-micronaut |
|  TikTok
https://www.tiktok.com/@codewithsamdev | Micronaut docs
https://docs.micronaut.io |
|  Instagram
https://www.instagram.com/codewithsamdev | Micronaut Data docs
https://micronaut-projects.github.io/micronaut-data/latest/guide/ |
|  Facebook
https://www.facebook.com/codewithsamdev | Spring Framework transactions
https://docs.spring.io/spring-framework/reference/data-access/transaction.html |

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. PostgreSQL is a trademark of the PostgreSQL Community Association. Micronaut is the property of its respective owner. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here.

Technical claims verified 2026-09-24 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.